generate a sequence diagram for a online shopping checkout process customer clicks buy now frontend sends request to backend backend checks user session with auth service if user not logged in return login page otherwise backend requests cart service for cart details then inventory service checks stock if stock availble payment service creates payment session with external payment gateway customer completes payment gateway sends webhook to backend backend verifies payment and creates order order service updates inventory and sends notification email if payment fails user gets error message and can retry payment

```
sequenceDiagram
    actor Customer
    participant FE as Frontend
    participant BE as Backend
    participant Auth as Auth Service
    participant Cart as Cart Service
    participant Inv as Inventory Service
    participant Pay as Payment Service
    participant PG as Payment Gateway
    participant Ord as Order Service

    Customer->>+FE: Click Buy Now
    FE->>+BE: Checkout Request
    BE->>+Auth: Validate Session
    Auth-->>-BE: Session Valid
    BE->>+Cart: Get Cart Details
    Cart-->>-BE: Cart Items
    BE->>+Inv: Check Stock
    Inv-->>-BE: Stock Available
    BE->>+Pay: Create Payment Session
    Pay->>+PG: Initialize Payment
    PG-->>-Pay: Payment Session
    Pay-->>-BE: Payment URL
    BE-->>-FE: Redirect to Payment
    FE-->>-Customer: Show Payment Page
    Customer->>+PG: Complete Payment
    PG-->>-Pay: Payment Success
    Pay-->>-BE: Payment Confirmation
    BE->>+Ord: Create Order
    Ord->>+Inv: Update Inventory
    Inv-->>-Ord: Inventory Updated
    Ord->>+BE: Order Created
    BE-->>-FE: Order Confirmation
    FE-->>-Customer: Show Success

    alt Payment Failed
        Customer->>+PG: Complete Payment
```

```
    PG-->>-Pay: Payment Failed
    Pay-->>-BE: Payment Failed
    BE-->>-FE: Error Message
    FE-->>-Customer: Show Error
    Customer->>FE: Retry Payment
  end
```

2

create sequence diagram for a food delivery order where customer places order from mobile app app sends order request to backend backend validates user and forwards request to order service order service checks restaurant availability if restaurant accepts then payment service processes payment while in parallel delivery service searches for rider if payment success and rider found order is confirmed otherwise cancel order and refund customer rider updates status picked up and delivered backend sends realtime updates to customer app during whole process

```
sequenceDiagram
  actor Customer
  participant App as Mobile App
  participant Backend as Backend
  participant OrderSvc as Order Service
  participant PaymentSvc as Payment Service
  participant DeliverySvc as Delivery Service
  participant Restaurant as Restaurant
  participant Rider as Rider

  Customer->>+App: Place order
  App->>+Backend: Send order request
  Backend->>+OrderSvc: Validate user & forward order
  OrderSvc->>+Restaurant: Check availability
  Restaurant-->>-OrderSvc: Accept/Reject

  alt Restaurant accepts
    OrderSvc->>+PaymentSvc: Process payment
    PaymentSvc-->>-OrderSvc: Payment result
```

```
    par Parallel processing
        OrderSvc->>+DeliverySvc: Find rider
        DeliverySvc-->>-OrderSvc: Rider found/not found
    and
        OrderSvc-->>-Backend: Payment status
    end

    alt Payment success and rider found
        OrderSvc-->>-Backend: Confirm order
        Backend-->>-App: Order confirmed
        Backend->>+Rider: Assign order
        Rider-->>-Backend: Accept

        Rider->>+Backend: Update picked up
        Backend-->>-App: Real-time update

        Rider->>+Backend: Update delivered
        Backend-->>-App: Real-time update
    else Payment failed or no rider
        OrderSvc-->>-Backend: Cancel order
        Backend-->>-App: Order cancelled
        Backend->>+PaymentSvc: Process refund
        PaymentSvc-->>-Backend: Refund processed
    end
else Restaurant rejects
    OrderSvc-->>-Backend: Order rejected
    Backend-->>-App: Restaurant unavailable
end

Backend-->>-App: Final status
App-->>-Customer: Show order status
```

3
draw a sequence diagram for a ai document chat platform user uploads pdf through frontend frontend sends file to upload service upload service stores file in cloud storage and notifies document processing service processing service extracts text generates embeddings and stores them in vector database user later asks question frontend sends query to backend backend retrieves relevant chunks from vector db sends context and question to llm api llm returns response backend sends answer to frontend if llm api timeout occurs retry two times then return fallback error message

```
sequenceDiagram
    actor User
    participant FE as Frontend
    participant US as Upload Service
    participant CS as Cloud Storage
    participant DPS as Document Processing Service
    participant VDB as Vector Database
    participant BE as Backend
    participant LLM as LLM API

    User->>+FE: Upload PDF file
    FE->>+US: Send file upload request
    US->>+CS: Store file in cloud storage
    CS-->>-US: File stored confirmation
    US->>+DPS: Notify new document
    DPS->>+DPS: Extract text from PDF
    DPS->>+VDB: Store text chunks
    VDB-->>-DPS: Chunks stored confirmation
    DPS-->>-US: Processing complete
    US-->>-FE: Upload successful
    FE-->>-User: Show uploaded document

    User->>+FE: Ask question
    FE->>+BE: Send query with context
    BE->>+VDB: Retrieve relevant chunks
    VDB-->>-BE: Return document chunks
    BE->>+LLM: Send context and question

    alt LLM API timeout
        LLM--xBE: Request timeout
        BE->>+LLM: Retry request 1
        alt Still timeout
            LLM--xBE: Request timeout
            BE->>+LLM: Retry request 2
            alt Still timeout
                LLM--xBE: Request timeout
                BE-->>-FE: Return fallback error
            else Success
                LLM-->>-BE: Response
```

```
        end
    else Success
        LLM-->>-BE: Response
    end
else Success
    LLM-->>-BE: Response
end

BE-->>-FE: Send answer
FE-->>-User: Display response
```

4

make a complicated architecture diagram for a smart city iot system thousands of sensors send temperature traffic and pollution data to iot gateway gateway forwards data to stream processing engine then data goes to real time dashboard and long term data lake anomaly detection ai service analyzes incoming data and sends alerts to emergency response system citizens use mobile app and web portal to view information include kafka message broker api gateway authentication service monitoring stack and disaster recovery backup

```
sequenceDiagram
    actor User
    participant FE as Frontend
    participant Auth as Auth Service
    participant DB as Database
    participant Notif as Notification Service

    User->>+FE: Enter email on forgot password
    FE->>+Auth: Request password reset
    Auth->>+DB: Verify account exists
    DB-->>-Auth: Account found
    Auth->>+DB: Generate reset token
```

```
DB-->>-Auth: Store token
Auth->>+Notif: Send reset email
Notif-->>-Auth: Email sent
Auth-->>-FE: Reset link sent
FE-->>-User: Show confirmation

User->>+FE: Click reset link
FE->>+Auth: Validate reset token
Auth->>+DB: Check token validity
DB-->>-Auth: Token valid
Auth-->>-FE: Token valid
FE->>+User: Show new password form
User->>+FE: Enter new password
FE->>+Auth: Submit new password
Auth->>+DB: Update password
DB-->>-Auth: Password updated
Auth->>+DB: Invalidate sessions
DB-->>-Auth: Sessions invalidated
Auth->>+Notif: Send confirmation email
Notif-->>-Auth: Email sent
Auth-->>-FE: Success response
FE-->>-User: Show success message

alt Token expired
    Auth->>+FE: Token expired
    FE->>+User: Request new reset link
end
```

5
generate sequence diagram for a ride sharing app customer requests ride mobile app sends request to backend backend queries location service and driver matching service driver matching service sends requests to nearby drivers in loop until one accepts if no driver accepts within 60 sec notify customer and suggest retry if driver accepts backend creates trip and sends details to both driver and customer during ride driver app periodically sends gps updates backend pushes live location updates to customer app after trip completion payment service charges user and sends receipt email

```
sequenceDiagram
    actor Customer
    participant App as Customer App
    participant Backend as Backend Server
    participant LocSvc as Location Service
    participant MatchSvc as Driver Matching Service
    participant Driver as Driver
    participant DriverApp as Driver App
    participant PaySvc as Payment Service
    participant EmailSvc as Email Service

    Customer->>App: Request Ride
    App->>+Backend: POST /ride/request
    Backend->>+LocSvc: Get Customer Location
    Backend->>+MatchSvc: Find Nearby Drivers
    MatchSvc->>+Driver: Request Ride
    alt Driver accepts
        Driver-->>-MatchSvc: Accept Ride
        MatchSvc-->>-Backend: Driver Found
        Backend->>Backend: Create Trip Record
        Backend->>App: Trip Confirmed
        Backend->>DriverApp: Trip Details
        App-->>Customer: Show Driver ETA
        DriverApp-->>Driver: Start Ride
    else No driver accepts in 60s
        Note over MatchSvc: 60s timeout
        MatchSvc-->>-Backend: No Driver Found
        Backend->>App: Notify No Driver
        App-->>Customer: Suggest Retry
    end
    loop During Ride
        DriverApp->>+Backend: Send GPS Update
        Backend->>App: Push Live Location
        Backend-->>-DriverApp: Acknowledge
    end
    Backend->>+PayS
```